

**Object-Oriented Programming in Practice: A Ride Sharing
System in C++ and Smalltalk**

Jalshree Khanal

[Institution Name]

[Course Name]

[Instructor Name]

July 15, 2026

Object-Oriented Programming in Practice: A Ride Sharing System in C++ and Smalltalk

This report describes a Ride Sharing System implemented twice—once in C++ and once in GNU Smalltalk—to demonstrate three core object-oriented programming (OOP) principles: encapsulation, inheritance, and polymorphism. Both versions share an identical design: a base *Ride* class; three subclasses, *StandardRide*, *PremiumRide*, and *SharedRide*, each pricing trips differently; a *Driver* class that accumulates completed rides; and a *Rider* class that accumulates requested rides. A short demonstration program stores several ride objects of different subclasses in one collection and calls `fare()` and `rideDetails()` on each polymorphically. The full source code, a build/run README, and execution screenshots are available at the project's GitHub repository: <https://github.com/jalshreekhana14-sketch/ride-sharing-oop>.

Encapsulation

C++ hides state using access specifiers: *Ride* keeps its fields `protected`, and *Driver* and *Rider* keep their ride lists (`assignedRides`, `requestedRides`) `private`, exposing them only through methods such as `addRide()` and `getDriverInfo()`. Encapsulation is therefore something the programmer must actively declare (Stroustrup, 2013). Smalltalk enforces the same guarantee unconditionally: an instance variable such as `assignedRides` is never visible outside the object at all, since the only way to interact with a Smalltalk object is by sending it a message (Goldberg & Robson, 1983). Encapsulation is not a stylistic choice in Smalltalk; it is a property of the object model itself.

Inheritance

C++ expresses inheritance with the `class Derived : public Base` syntax, and subclass constructors explicitly forward shared state to the base constructor—for example, *StandardRide*'s constructor delegates to `Ride(rideID, pickup, dropoff, distance)` in its initializer list. Smalltalk expresses the same relationship through a class-creation message sent to the superclass, `Ride subclass: #StandardRide ...`, reflecting Smalltalk's more dynamic style, in which even class definitions are messages sent to

existing objects (Goldberg & Robson, 1983). Both implementations produce an identical single-inheritance hierarchy rooted at *Ride*.

Polymorphism

C++ requires `fare()` and `rideDetails()` to be declared `virtual` in the base class, with `fare()` as a pure virtual function, enabling dynamic dispatch when objects are accessed through a `std::shared_ptr<Ride>` (Stroustrup, 2013). Iterating over a vector of such pointers and calling `ride->fare()` invokes the correct override for each concrete subclass at runtime. Smalltalk has no `virtual` keyword because every method call is a dynamically dispatched message by default: sending `fare` to any *Ride* causes the runtime to look up the method starting at the receiver's actual class (Free Software Foundation, 2023). *StandardRide*, *PremiumRide*, and *SharedRide* each simply define their own `fare` method, and polymorphism follows automatically.

Discussion

The comparison highlights a difference in philosophy rather than capability. C++ is statically typed and compiled, so it makes OOP guarantees opt-in and enforces them at compile time, giving the programmer fine-grained control over which behaviors are polymorphic, encapsulated, or inherited. Smalltalk is dynamically typed and purely message-based, so the same guarantees are unconditional consequences of its object model. Despite this difference, both languages arrive at the same object-oriented design for the Ride Sharing System, suggesting that encapsulation, inheritance, and polymorphism are language-independent concepts whose enforcement mechanisms simply differ.

Conclusion

Implementing the same Ride Sharing System in C++ and Smalltalk shows that encapsulation, inheritance, and polymorphism can be realized through different mechanisms while producing equivalent designs: C++ favors explicit, compiler-checked declarations, while Smalltalk favors implicit guarantees built into a pure message-passing object model.

References

Free Software Foundation. (2023). *GNU Smalltalk user's guide*.

https://www.gnu.org/software/smalltalk/manual-base/html_node/

Goldberg, A., & Robson, D. (1983). *Smalltalk-80: The language and its implementation*.

Addison-Wesley.

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.

Appendix: Code and Sample Output

Figure 1

C++ — Ride Base Class and StandardRide / PremiumRide Subclasses

RideSharingSystem.cpp

```
10 // -----
11 // Ride: base class. Holds core ride details and defines the
12 // polymorphic interface (fare() and rideDetails()) that every
13 // subclass must specialize.
14 // -----
15 class Ride {
16 protected:
17     // Encapsulation: fields are protected/private, only reachable
18     // through accessors or the constructor.
19     int rideID;
20     std::string pickupLocation;
21     std::string dropoffLocation;
22     double distance; // in miles
23
24 public:
25     Ride(int rideID, const std::string& pickupLocation,
26          const std::string& dropoffLocation, double distance)
27         : rideID(rideID), pickupLocation(pickupLocation),
28           dropoffLocation(dropoffLocation), distance(distance) {}
29
30     virtual ~Ride() = default;
31
32     // Polymorphism: subclasses override this to price rides differently.
33     virtual double fare() const = 0;
34
35     // Polymorphism: subclasses may customize the printed summary too.
36     virtual void rideDetails() const {
37         std::cout << "Ride #" << rideID
38                   << " | " << pickupLocation << " -> " << dropoffLocation
39                   << " | Distance: " << std::fixed << std::setprecision(1)
```

```

40         << distance << " mi"
41
42         << " | Fare: $" << std::fixed << std::setprecision(2)
43
44         << fare() << std::endl;
45     }
46
47     // Read-only accessors keep the internal state encapsulated.
48     int getRideID() const { return rideID; }
49     double getDistance() const { return distance; }
50     std::string getPickupLocation() const { return pickupLocation; }
51     std::string getDropoffLocation() const { return dropoffLocation; }
52 };
53
54 // -----
55 // StandardRide: everyday, budget-friendly ride type.
56 // -----
57
58 class StandardRide : public Ride {
59 private:
60     static constexpr double BASE_FARE = 2.00;
61     static constexpr double RATE_PER_MILE = 1.25;
62
63 public:
64     StandardRide(int rideID, const std::string& pickup,
65                  const std::string& dropoff, double distance)
66         : Ride(rideID, pickup, dropoff, distance) {}
67
68     double fare() const override {
69         return BASE_FARE + (distance * RATE_PER_MILE);
70     }
71
72     void rideDetails() const override {
73         std::cout << "[Standard] ";
74         Ride::rideDetails();
75     }
76 }

```

```
73 };
```

```
74
```

```
75 //
```

Note. Shows encapsulated fields (protected), the pure virtual `fare()` declared in *Ride*, and each subclass's overridden `fare()/rideDetails()` (inheritance and polymorphism).

Figure 2

C++ — Driver Class Demonstrating Encapsulation

```
RideSharingSystem.cpp – Driver (encapsulation)

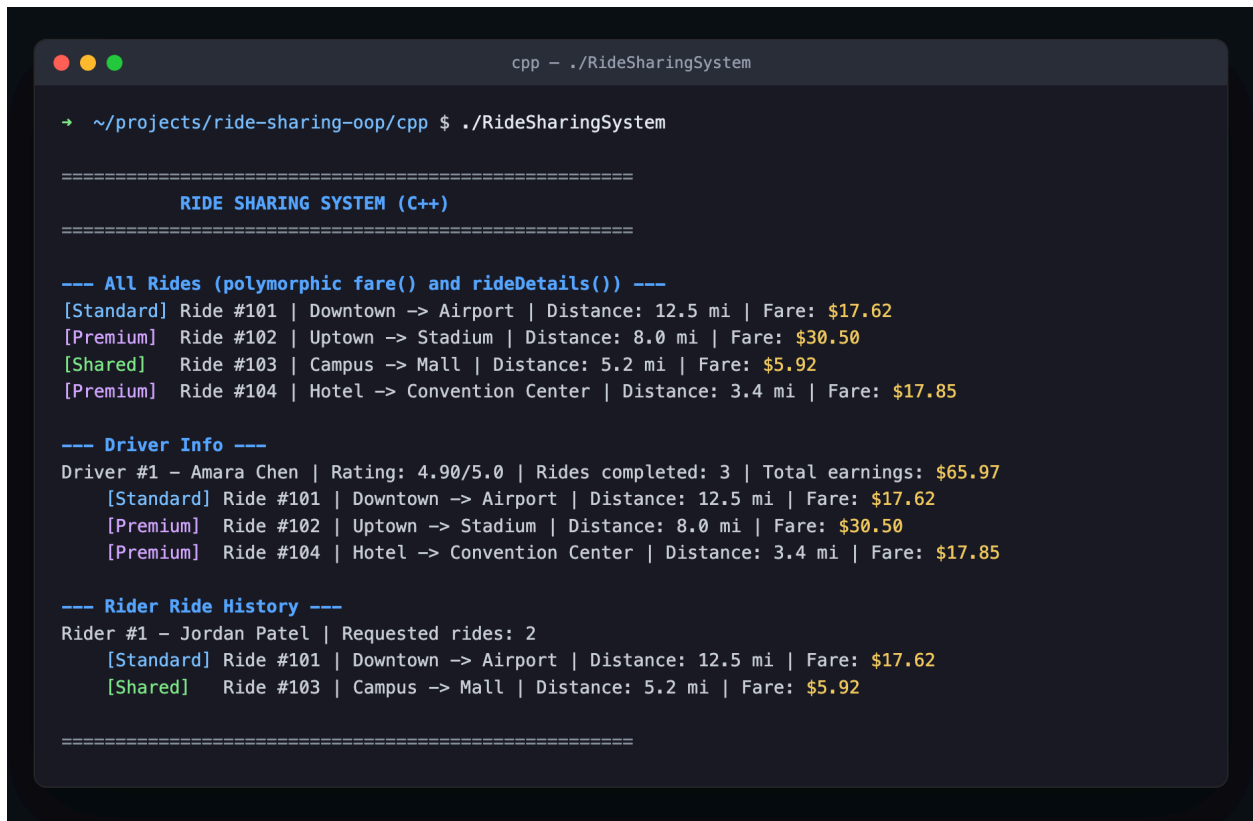
128 class Driver {
129 private:
130     int driverID;
131     std::string name;
132     double rating;
133     std::vector<std::shared_ptr<Ride>> assignedRides; // private list
134
135 public:
136     Driver(int driverID, const std::string& name, double rating)
137         : driverID(driverID), name(name), rating(rating) {}
138
139     void addRide(const std::shared_ptr<Ride>& ride) {
140         assignedRides.push_back(ride);
141     }
142
143     double totalEarnings() const {
144         double total = 0.0;
145         for (const auto& ride : assignedRides) {
146             total += ride->fare(); // polymorphic call
147         }
148         return total;
149     }
150
151     void getDriverInfo() const {
152         std::cout << "Driver #" << driverID << " - " << name
153             << " | Rating: " << rating << "/5.0"
154             << " | Rides completed: " << assignedRides.size()
155             << " | Total earnings: $" << std::fixed
156             << std::setprecision(2) << totalEarnings() << std::endl;
157         for (const auto& ride : assignedRides) {
```

```
158         std::cout << " ";
159         ride->rideDetails(); // polymorphic call
160     }
161 }
162 };
163
164 // -----
165 // Rider: keeps a private history of requested rides.
166 // -----
```

Note. `assignedRides` is private and reachable only through `addRide()` and `getDriverInfo()`; `totalEarnings()` and `getDriverInfo()` call `ride->fare()` and `ride->rideDetails()` polymorphically.

Figure 3

C++ — Sample Program Output



```
cpp - ./RideSharingSystem

→ ~/projects/ride-sharing-oop/cpp $ ./RideSharingSystem

=====
                RIDE SHARING SYSTEM (C++)
=====

--- All Rides (polymorphic fare() and rideDetails()) ---
[Standard] Ride #101 | Downtown -> Airport | Distance: 12.5 mi | Fare: $17.62
[Premium]   Ride #102 | Uptown -> Stadium | Distance: 8.0 mi | Fare: $30.50
[Shared]    Ride #103 | Campus -> Mall | Distance: 5.2 mi | Fare: $5.92
[Premium]   Ride #104 | Hotel -> Convention Center | Distance: 3.4 mi | Fare: $17.85

--- Driver Info ---
Driver #1 - Amara Chen | Rating: 4.90/5.0 | Rides completed: 3 | Total earnings: $65.97
  [Standard] Ride #101 | Downtown -> Airport | Distance: 12.5 mi | Fare: $17.62
  [Premium]   Ride #102 | Uptown -> Stadium | Distance: 8.0 mi | Fare: $30.50
  [Premium]   Ride #104 | Hotel -> Convention Center | Distance: 3.4 mi | Fare: $17.85

--- Rider Ride History ---
Rider #1 - Jordan Patel | Requested rides: 2
  [Standard] Ride #101 | Downtown -> Airport | Distance: 12.5 mi | Fare: $17.62
  [Shared]    Ride #103 | Campus -> Mall | Distance: 5.2 mi | Fare: $5.92

=====
```

Note. Output of `./RideSharingSystem`, showing polymorphic `fare()`/`rideDetails()` results for all ride types, driver earnings, and rider history.

Figure 4

Smalltalk — Ride, StandardRide, and PremiumRide Class Definitions

RideSharingSystem.st

```
45 Ride extend [
46     setRideID: anID pickup: aPickup dropoff: aDropoff distance: aDistance [
47         rideID := anID.
48         pickupLocation := aPickup.
49         dropoffLocation := aDropoff.
50         distance := aDistance.
51     ]
52
53     "Read-only accessors – the only way outside code can see the state."
54     rideID [ ^rideID ]
55     pickupLocation [ ^pickupLocation ]
56     dropoffLocation [ ^dropoffLocation ]
57     distance [ ^distance ]
58
59     "Every subclass MUST override fare – this is the polymorphic hook."
60     fare [ ^self subclassResponsibility ]
61
62     rideDetails [
63         Transcript showCr: 'Ride #', rideID printString, ' | ',
64             pickupLocation, ' -> ', dropoffLocation,
65             ' | Distance: ', (NumberFormatter decimal: distance places: 1), ' mi',
66             ' | Fare: $', (NumberFormatter decimal: self fare places: 2).
67     ]
68 ]
69
70 "=====
71 StandardRide: everyday, budget-friendly ride type.
72 ====="
73 Ride subclass: #StandardRide
74     instanceVariableNames: ''
```

```

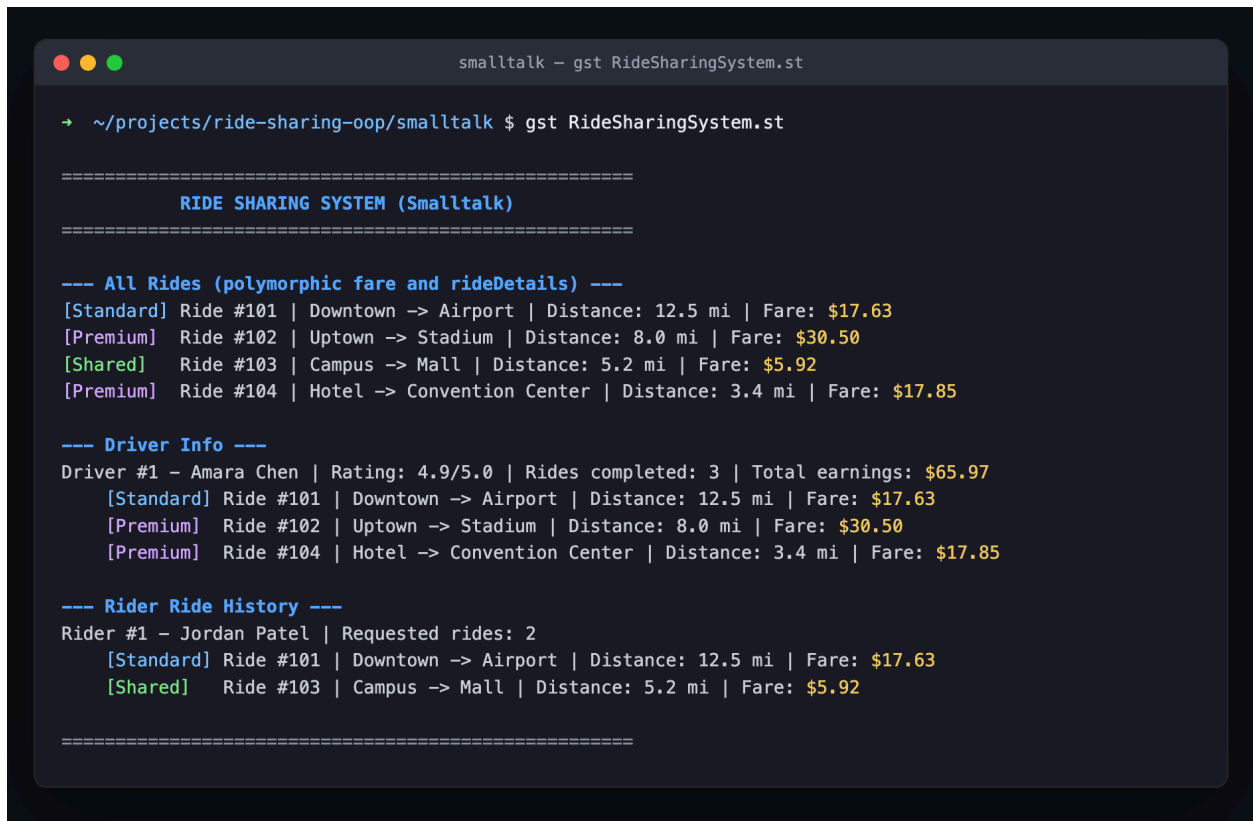
75     classVariableNames: ''
76     poolDictionaries: ''
77     category: 'RideSharing'.
78
79 StandardRide extend [
80     fare [
81         ^2.00 + (distance * 1.25)
82     ]
83
84     rideDetails [
85         Transcript show: '[Standard] '.
86         super rideDetails.
87     ]
88 ]
89
90 "=====
91 PremiumRide: higher base fare, higher per-mile rate, plus a luxury
92 surcharge - overrides fare differently than StandardRide.
93 ====="
94 Ride subclass: #PremiumRide
95     instanceVariableNames: ''
96     classVariableNames: ''
97     poolDictionaries: ''
98     category: 'RideSharing'.
99

```

Note. Instance variables have no accessor unless explicitly defined (encapsulation by default); `Ride subclass: #StandardRide` creates the subclass (inheritance); each subclass overrides `fare` (polymorphism).

Figure 5

Smalltalk — Sample Program Output



```
smalltalk - gst RideSharingSystem.st

→ ~/projects/ride-sharing-oop/smalltalk $ gst RideSharingSystem.st

=====
                RIDE SHARING SYSTEM (Smalltalk)
=====

--- All Rides (polymorphic fare and rideDetails) ---
[Standard] Ride #101 | Downtown -> Airport | Distance: 12.5 mi | Fare: $17.63
[Premium]  Ride #102 | Uptown -> Stadium | Distance: 8.0 mi | Fare: $30.50
[Shared]   Ride #103 | Campus -> Mall | Distance: 5.2 mi | Fare: $5.92
[Premium]  Ride #104 | Hotel -> Convention Center | Distance: 3.4 mi | Fare: $17.85

--- Driver Info ---
Driver #1 - Amara Chen | Rating: 4.9/5.0 | Rides completed: 3 | Total earnings: $65.97
  [Standard] Ride #101 | Downtown -> Airport | Distance: 12.5 mi | Fare: $17.63
  [Premium]  Ride #102 | Uptown -> Stadium | Distance: 8.0 mi | Fare: $30.50
  [Premium]  Ride #104 | Hotel -> Convention Center | Distance: 3.4 mi | Fare: $17.85

--- Rider Ride History ---
Rider #1 - Jordan Patel | Requested rides: 2
  [Standard] Ride #101 | Downtown -> Airport | Distance: 12.5 mi | Fare: $17.63
  [Shared]   Ride #103 | Campus -> Mall | Distance: 5.2 mi | Fare: $5.92

=====
```

Note. Output of `gst RideSharingSystem.st`, mirroring the C++ program's behavior via message sends instead of virtual function calls.